

CS 2620 Design Exercise 3 Engineering Notebook (Replication Project)  
Savanna Coffel & Ian Moore

Thursday, March 13, 2025

We have discussed the requirements laid out in the specification for the assignment and have agreed to start on an initial implementation of the distributed system. We agreed that some of the most critical issues here will be the implementation for tracking the different server hosts and identifiers especially across different machines, and keeping this current as servers come offline or online (sometimes unexpectedly due to crashes, network failures, etc. — there are many possible unforeseeable circumstances).

We have agreed on a baseline level of implementation. We intend to target the ability to add additional hosts. However, the replication is simple enough to implement with just a fixed initial number of three hosts, and then allowing additional replication after we have implemented this minimum level of functionality. The extension to allow the addition of new hosts will allow us to run this across multiple machines as required.

Hence we intend to create a startup script that will instantiate three servers on a single machine, all replicas of each other.

We will use the Raft consensus algorithm to conduct leader election.

We justify our choice of the Raft consensus algorithm for its simplicity, ease of use, and adoption in proven production settings. We like that Raft simplifies the flow of data through a single leader that manages the replication. This seems to us like an intuitive implementation and a reasonable way of managing the data flow.

Friday, March 14, 2025

We met to discuss the initial implementation and the lingering issues. The current implementation uses heartbeats, leader election using Raft consensus, and tracking of hosts as hardcoded config. This last part is a sticky issue for us.

Basically, in the current setup, each server sends out heartbeats that let the other servers know it is still alive and part of the pool of replicas. If the leader fails to send out heartbeats, then this triggers the Raft consensus algorithm leader election process.

We are targeting the 'extra credit' implementation such that we are able to add (arbitrarily many, we suppose though within reason of the limits of modern computing) new servers to the pool of replicas even after we have started the initial set, as stated in our previous entry from yesterday.

This will require some further extension of the codebase. We will need to create an extensible list of connected servers, and then manage how servers will join and exit the list in various scenarios such as startup, failure, network outage, etc.

We have decided to use Zookeeper/Kazoo to manage our knowledge of the connected servers.

Zookeeper is a proven library for managing server information, used in many large scale distributed systems, and is an appropriate tool for our application as it will allow us to register the replicas and keep a current list of them. One machine will run the Zookeeper registry.

We like the Kazoo package for interacting with Zookeeper as it provides a nice simple interface which will interact nicely with our Python code.

To recap the design of our scripts, we have one script, `server.py`, which spawns in the initial set of 3 servers which are replicas, and creates leader election of a leader via the Raft consensus algorithm.

Monday, March 24, 2025

We have returned from spring break.

We are satisfied with our initial implementation on a high level.

We designed the system so that the individual servers send heartbeats. If there is a missed heartbeat by the leader, then this invokes leader election via the Raft consensus algorithm.

One issue that came up in our testing was that if we try to “swap out” the leader, then we observe weird behaviors from the application. We have decided that as a design decision it makes little sense to consider the case of “swapping out” a leader. We consider the leader to be chosen through our standard leader election process (which we have again implemented via the Raft consensus algorithm) and we do not allow a new server to arbitrarily decide to insert itself as a leader. A server can only become leader through our election process via the Raft consensus algorithm. This greatly simplifies the inter-server communications among the replicas and ensures that the leader choice is controlled across our distributed system. We therefore believe this is an intelligent or at least reasonable design choice to maintain the elegance, maintainability, and simplicity of our distributed system.

We have also noted odd pathologies in our testing if there are overlapping server ids and server ports. We have decided that as a design decision we will implement logic in our code ensuring that servers cannot be added if there is a server already registered in Zookeeper with the same server id or port. This is simply a design decision which we believe is reasonable to maintain the simplicity of our codebase and ensure smooth operation of the replica registration and management. There is no reason why it would make sense to have replicas registered with the same identifier or on overlapping ports. We will include this critical design decision in our readme documentation to ensure that users are aware of this, however we also created intuitive error messages that will be provided to users if they attempt to register a server on an already existing port or with an already existing name. This intuitive error message will provide plenty of notice to the user. Indeed we have also tested these error messages and have observed that they do occur as desired and the error handling prevents the spawning of new servers with the same server id or port as one already existing, as desired.

Tuesday, March 25, 2025

We would like to discuss our **approach to testing**. We have created both **comprehensive unit tests** as well as **integration tests**. We have evaluated our unit tests to ensure the coverage level of the codebase is high, and we generated a complete coverage report via `pytest` which is accessible via the terminal by running our tests.

Our unit test suite overall relies on some sophisticated mocking and imitation of the various components of the distributed system to ensure that the units of our codebase behave as intended and everything functions appropriately.

Overall we are very satisfied with the level of test coverage that we have obtained and we believe our tests are comprehensive enough for us to ascertain that our end product meets the requirements of the specification.

We have also conducted our own manual user tests of course as well. We are pleased that the chat app continues functioning appropriately even after our refactor that adds replication across multiple servers. We also tested many of the failures, outages, disconnects, quits, etc. that we would expect in deployment of such a distributed system and were pleased to see that our error handling has been designed sufficiently to account for these types of scenarios.

In reflection, we have found that the design choices we made previously have paid off to create a cohesive, clear, easy to maintain, and robust codebase. We find that the elegance of having the Raft consensus algorithm simply manage replication and leader elections and letting new servers join the pool of replicas but only via a unique port and host as well as our other decision decisions has led to the success of this implementation.